UNIVERSITATEA TEHNICĂ A MOLDOVEI

Технический университет Молдовы

Факультет вычислительной техники,
информатики и микроэлектроники

Департамент программной инженерии и автоматики

Программирование мобильных приложений

Programarea aplicațiilor mobile

Отчёт по лабораторной работе № 5

Интеграция с собственным backend
Индивидуальная тема RunBaza

Выполнил: Hamuev Oleg

Группа: CR-232

Проверил: Leah A.

Учебный год 2026–2027

Кишинёв 2026

1 Цель и результат работы

В Л5 RunBaza подключён к собственному backend на TypeScript и Hono. Регистрация и вход проверяются сервером, записи сохраняются в SQLite, а Flutter получает их через HTTP-реализации интерфейсов из Л4. Тема приложения: календарь забегов и личное планирование поездок для Молдовы и Румынии.

Основная задача работы состоит в замене источника данных без переписывания экранов. Эта замена зафиксирована отдельным коммитом. После неё отдельным этапом добавлены поиск на сервере и постраничная загрузка каталога.

Требование Л5	**Результат**
Backend и сущности темы	Hono, SQLite, JWT; races, plans, offers, requests, profiles
Настоящий вход	POST register/login, token в shared_preferences, logout
HTTP-репозитории	Пять методов CRUD, fromJson/toJson, серверные ID
Ошибки и 401	Три слоя Err; перевод кодов; очистка сессии и переход на вход
Замена mock	DI в main.dart; runtime mock и диагностический switch удалены
Поиск и страницы	28 демонстрационных событий; страницы 20 + 8; search и country
Upload	POST /files до 5 MiB и загрузка по ID проверены на API
Проверка	58 HTTP-проверок, тест перезапуска БД, Flutter и реальный iOS

Выбранная реализация

Вместо языковых примеров курса реализован самостоятельный сервер TypeScript/Hono. Проверки проекта не являются официальным набором conformance из kit: его файлы не предоставлены, поэтому результат 37/37 не заявляется. В текущей теме нет пользовательской загрузки изображений: иллюстрации маршрутов рисуются приложением.

2 Архитектура и жизненный цикл сессии

Экраны вызывают методы моделей. Модели зависят от интерфейсов Repository, а конкретные HttpRepository создаются в корне композиции. Один ApiClient отвечает за сетевой запрос, Bearer token, deadline и разбор ответа.

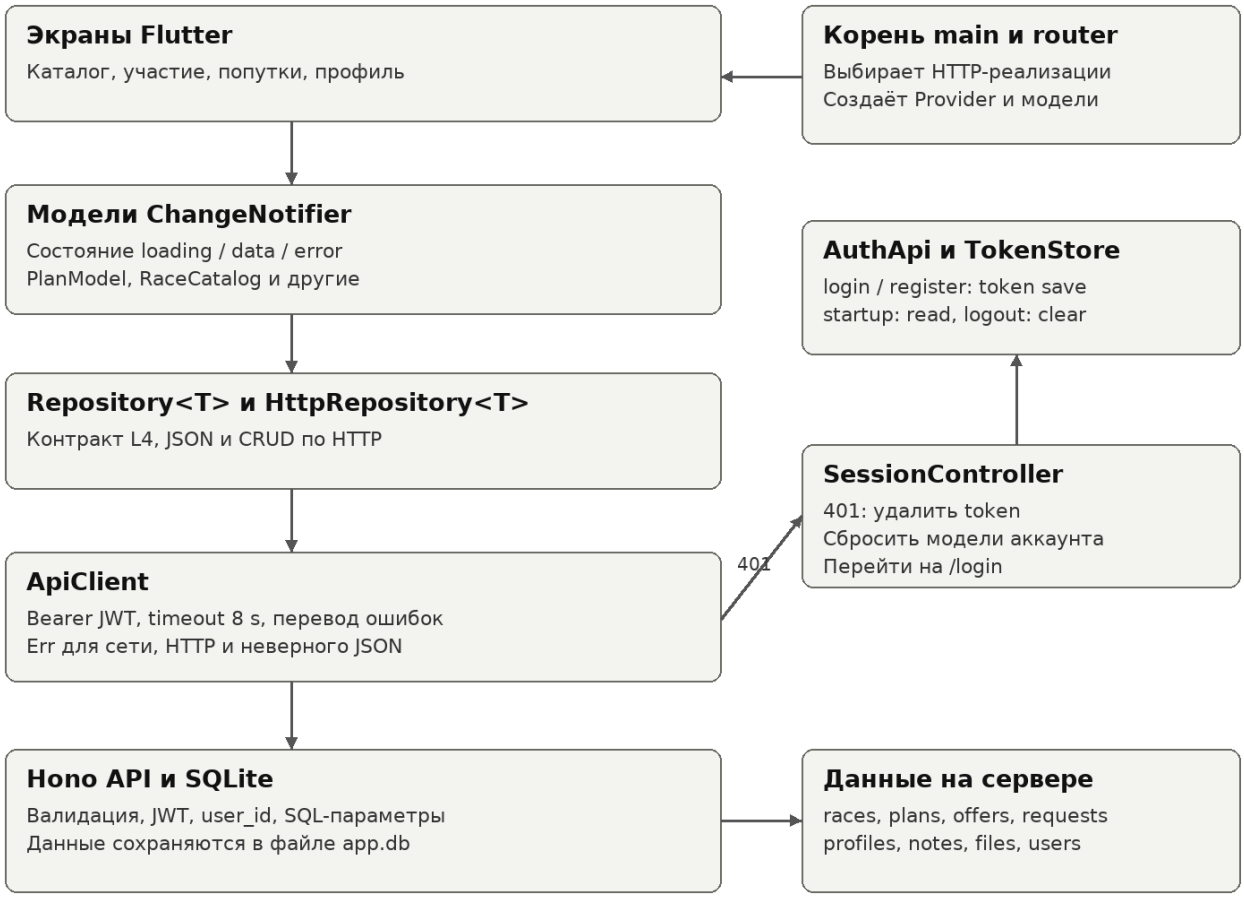


Рисунок 1. Архитектура RunBaza после подключения собственного API.

При замене источника экраны данных и модели состояния не изменились: они уже работали с Future<Result<T>> и не зависели от mock. Исключения: формы входа/регистрации и профиль с удалённым диагностическим переключателем.

От запроса до выхода из сессии

Успешный auth сохраняет JWT, затем SessionController открывает защищённые маршруты. При запуске TokenStore читает сохранённый token. Ответ 401 на защищённый запрос удаляет его, увеличивает generation сессии и уведомляет router. Старые модели аккаунта уничтожаются; следующий вход создаёт новые модели. Пароль не записывается в prefs.

3 Сервер и контракт данных

Сервер находится в Lab2/server. index.ts запускает HTTP на 127.0.0.1:8080. database.ts создаёт таблицы и начальный каталог, schema.ts проверяет входные поля, app.ts регистрирует маршруты. SQLite хранит файл data/app.db; повторный запуск не сбрасывает записи.

Маршрут	Назначение
POST /auth/register	Email, password, name; 201 и token; повтор email даёт 409
POST /auth/login	Проверка пароля; 200 и token либо 401
/races	Общий seed только для чтения и собственные события
/plans	Личное участие: raceId, distance, pace, reminderDays, visibility
/offers и /requests	Личные записи водителя и запросы пассажира
/profiles	Профиль создаётся при регистрации, изменяется через PUT
/notes	Базовая сущность для проверки CRUD-контракта
/files	Multipart upload и скачивание файла владельцем

Для коллекций доступны GET и POST; для /:id доступны GET, PUT и DELETE. Успешные статусы: чтение/изменение 200, создание 201, удаление 204 без тела. В ответе списка: items, page, limit, total. ID выдаёт сервер; доменные ID имеют формат UUID.

Хранение и доступ

В доменных таблицах есть id, user_id, race_id, JSON data и search_text. Внешние ключи связывают записи с аккаунтом и забегом. Параметры SQL передаются отдельно от текста запроса. Чужая приватная запись возвращает 404. Пароль хранится как scrypt-хеш с отдельной случайной солью, JWT подписан HS256 и действует 24 часа.

План, профиль и записи попуток в этой лабораторной принадлежат аккаунту. Публичное бронирование мест и публикация плана ещё не реализованы. Учебный seed не является актуальным расписанием реальных соревнований.

4 Проверка backend

Команда pnpm test поднимает настоящий HTTP-сервер на свободном порту и выполняет 58 проверок. Отдельный тест создаёт временную SQLite-базу, перезапускает сервер и проверяет сохранность записи и отказ старого token после смены JWT_SECRET.

```
cd Lab2/server
pnpm check
pnpm test

PASS 54 file bytes and safe download headers
PASS 55 file privacy
PASS 56 upload size limit
PASS 57 malformed JSON envelope
PASS 58 bad token signature
PASS 58/58 RunBaza project checks (not the course kit suite)
✔ HTTP contract against a real listening Hono server (163.629375ms)
✔ SQLite restart preserves records and changed JWT secret rejects the old session
  (31.2735ms)
ℹ tests 2
ℹ pass 2
ℹ fail 0
ℹ duration_ms 416.385916
```

Листинг 1. Выдержка реального вывода. Полный журнал: Lab5/evidence/backend-tests.txt. Это собственный набор проекта, не официальный conformance курса.

Группа	Что проверяется
Auth	Регистрация, повтор email, неверный пароль, отсутствие/ошибка/истечение token
CRUD	Создание, чтение, обновление, пустое тело 204, неизвестный ID
Изоляция	Чужие notes, plans, profile и files недоступны по ID
Предметная область	Связь с race, допустимая дистанция, уникальность участия
Поиск и страницы	Unicode search, page/limit/country, неверные параметры
Upload	Multipart, байты при скачивании, предел 5 MiB, заголовки

Граница доказательства

Автоматические проверки относятся к зафиксированному коду и локальному окружению. Они не доказывают соответствие недоступному openapi.yaml и официальному набору курса. Сервер предназначен для локальной защиты, без production deploy, восстановления пароля и ограничения частоты запросов.

5 Сессия curl на сущности участия

Запросы выполнены к тестовому экземпляру на порту 8090. Обычный запуск использует 8080. JWT скрыт в журнале; UUID оставлен как подтверждение того, что ID присвоил сервер.

```
$ curl -X POST http://127.0.0.1:8090/auth/register -H "Content-Type: application/json" -d '{"email":
  "curl-5e728b02-f168-4f7e-aa2c-2df90a336c63@example.com", "password": "runbaza26", "name": "Curl
  Runner"}'
HTTP 201
{"token": "[JWT redacted]", "user": {"id": 8, "email":
  "curl-5e728b02-f168-4f7e-aa2c-2df90a336c63@example.com", "name": "Curl Runner"}}

$ curl -X GET http://127.0.0.1:8090/plans
HTTP 401
{"error": {"code": "UNAUTHORIZED", "message": "Authentication required"}}

$ curl -X POST http://127.0.0.1:8090/plans -H "Content-Type: application/json" -d '{"raceId": "r1",
  "reminderDays": 2, "distance": 10, "pace": "5:40", "visibility": "private"}' -H "Authorization:
  Bearer $TOKEN"
HTTP 201
{"id": "6e91bc21-9e74-4cea-bf1e-be3a5b096f91", "raceId": "r1", "reminderDays": 2, "distance": 10,
  "pace": "5:40", "visibility": "private"}

$ curl -X GET http://127.0.0.1:8090/plans?search=5%3A40 -H "Authorization: Bearer $TOKEN"
HTTP 200
{"items": [{"id": "6e91bc21-9e74-4cea-bf1e-be3a5b096f91", "raceId": "r1", "reminderDays": 2,
  "distance": 10, "pace": "5:40", "visibility": "private"}], "page": 1, "limit": 20, "total": 1}
```

Листинг 2. Регистрация, 401 без Bearer, создание участия и поиск. Полный журнал с PUT и DELETE: evidence/curl-session.txt.

Поиск 5:40 выполняется на сервере и возвращает созданное участие. После DELETE сервер отвечает 204 с пустым телом; клиент возвращает Ok<void> без jsonDecode.

6 Настоящая регистрация и вход

Формы сохраняют валидацию ЛЗ. На время запроса кнопка блокируется. Регистрация отправляет имя, email и пароль, создаёт аккаунт и профиль. Вход сравнивает пароль на сервере; локально остаётся только token. Индикатор входа показывается минимум три секунды.

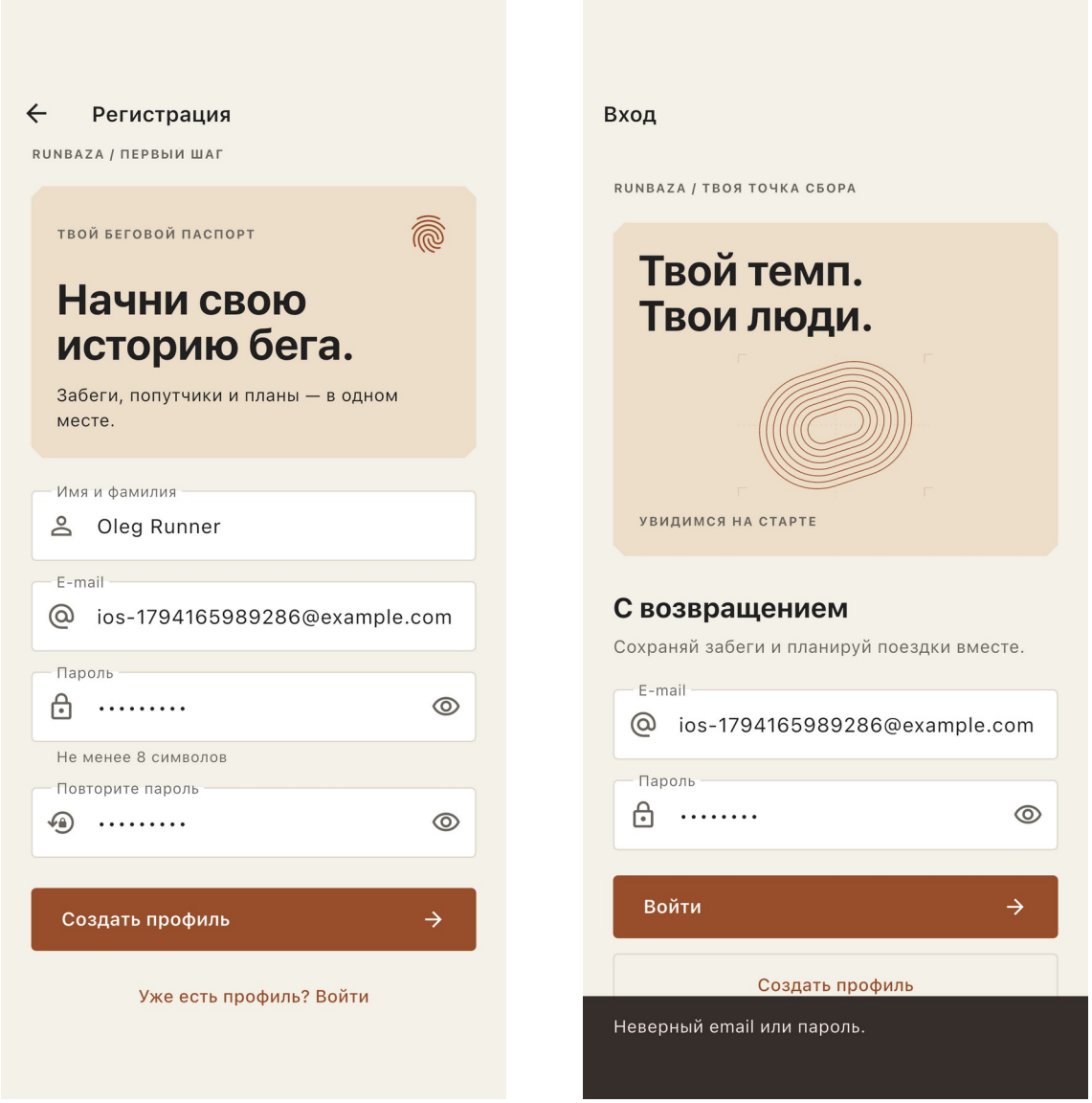


Рисунок 2. Реальная регистрация и переведённая серверная ошибка входа.

Неверный пароль даёт HTTP 401 в auth endpoint, но не запускает очистку чужой активной сессии: auth-запрос отмечен authenticated: false. Сообщение «Неверный email или пароль» выбирается по стабильному коду ошибки.

7 HTTP и три слоя ошибок

HttpRepository<T> реализует контракт из Л4, а повторяющаяся обработка ошибок вынесена в ApiClient.request<T>. Пример чтения списка и проверки результата:

```
@override
  Future<Result<List<T>>> fetchAll({
    int page = 1,
    int limit = 20,
    String? search,
    String? country,
  }) => api.request(
    'GET',
    path,
    successStatus: 200,
    query: {
      'page': '$page',
      'limit': '$limit',
      if (search != null && search.isNotEmpty) 'search': search,
      if (country != null && country.isNotEmpty) 'country': country,
    },
    decode: (json) {
      final envelope = json as Map<String, dynamic>;
      return (envelope['items'] as List)
          .map((item) => fromJson(item as Map<String, dynamic>))
          .toList();
    },
  );
```

```
if (response.statusCode == 401 && authenticated) {
        await _expire(token);
        return const Err('Сессия завершена. Войдите снова.');
      }
      if (response.statusCode != successStatus) {
        return Err(apiMessage(response, signingIn: signingIn));
      }
      // A successful DELETE has no JSON body.
      final json = successStatus == 204
          ? null
          : jsonDecode(utf8.decode(response.bodyBytes));
      return Ok(decode(json));
    } on SocketException {
      return const Err('Не удалось подключиться к серверу. Повторите попытку.');
    } on http.ClientException {
      return const Err('Не удалось подключиться к серверу. Повторите попытку.');
    } on TimeoutException {
      return const Err('Сервер не отвечает. Повторите попытку.');
    } on FormatException {
      return const Err('Сервер вернул некорректные данные.');
    } on TypeError {
      return const Err('Сервер вернул некорректные данные.');
    }
```

Первый слой: транспорт и timeout. Второй: неуспешный statusCode и apiMessage. Третий: неверный JSON или тип поля. Все они превращаются в Err, который уже умеет показывать UI Л4. Общего catch для маскировки программных ошибок нет.

8 Сохранение участия и профиля

Форма участия отправляет POST /plans. Повторное сохранение того же забега использует PUT существующего ID. Сервер проверяет принадлежность записи и допустимую дистанцию. После Ok модель обновляет план и общие счётчики. Профиль работает через отдельный HTTP-репозиторий.

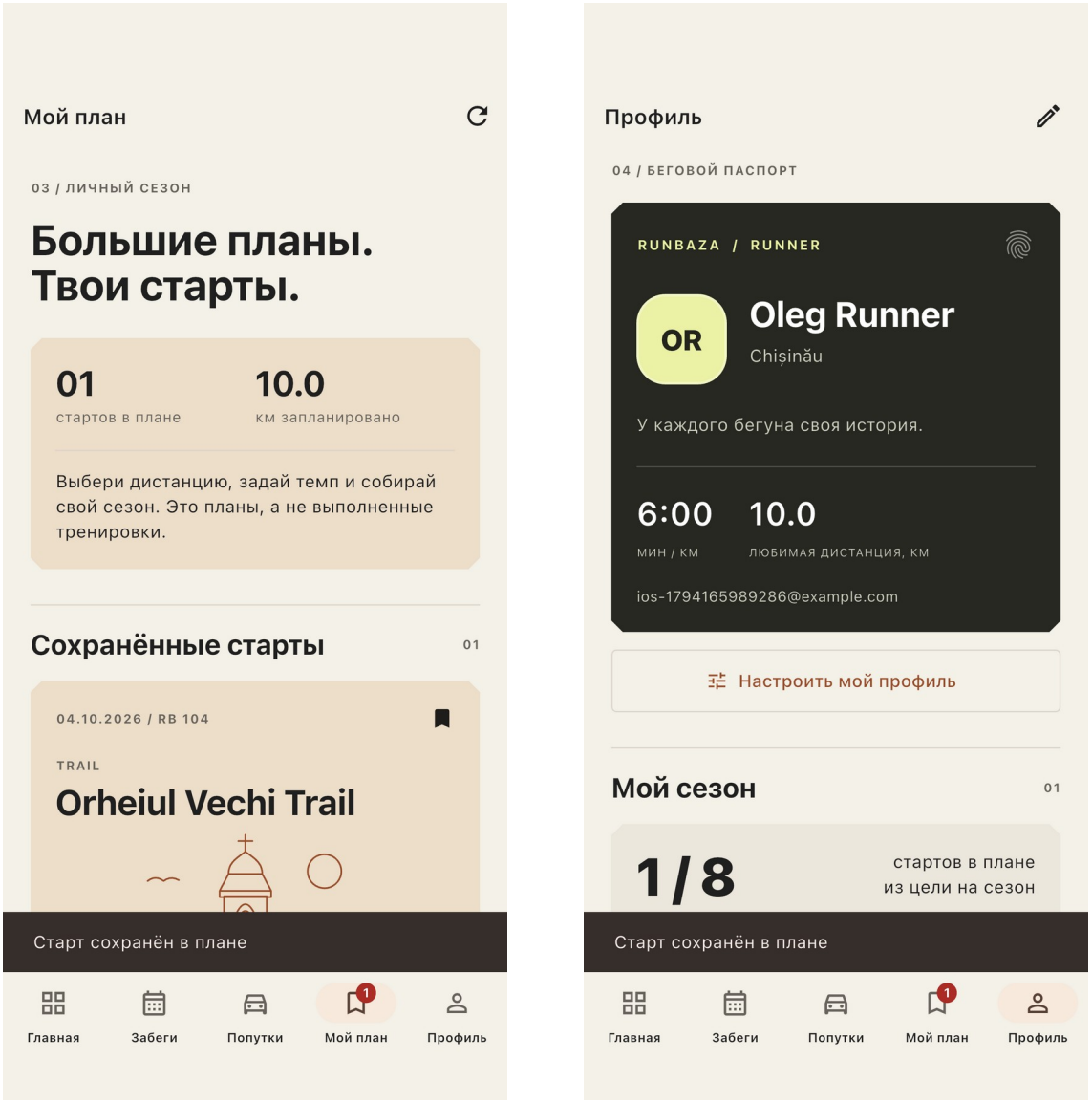


Рисунок 3. Участие после ответа API и профиль того же аккаунта.

При перезапуске Flutter список запрашивается заново, а сохранённая запись остаётся в SQLite. Изменение темы и параметров профиля также проходит через API. Удаление участия удаляет личную запись, а не забег общего каталога.

9 Поиск и постраничная загрузка

fetchAll получил необязательные именованные параметры page, limit, search и country. Старый вызов fetchAll() остался корректным. Каталог запрашивает страницу из 20 записей; прокрутка догружает оставшиеся 8. На одном экране не запускаются два одинаковых loadMore.

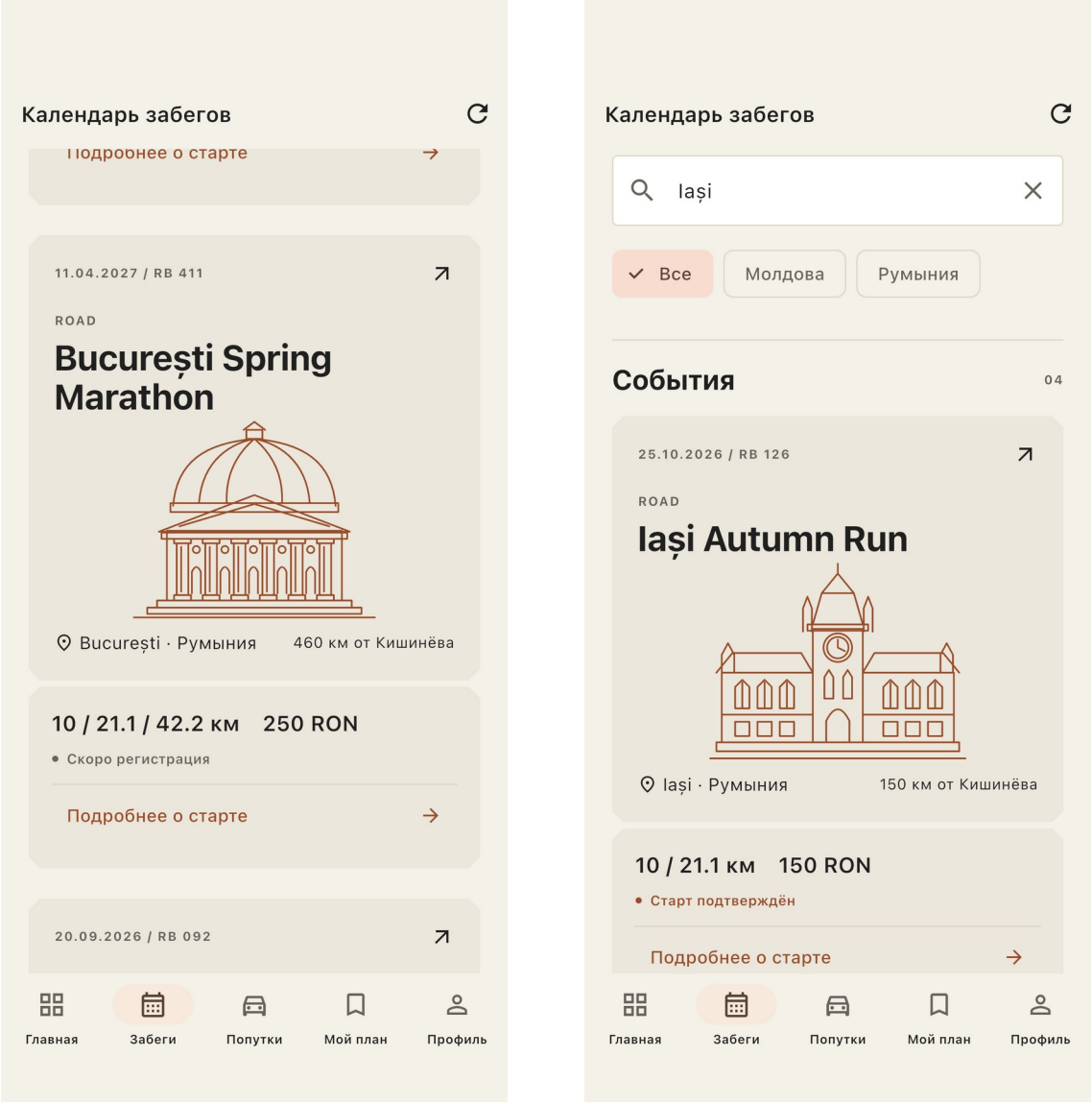


Рисунок 4. Каталог после догрузки и результат поиска Iași на сервере.

Поиск отправляется после паузы ввода 400 мс или подтверждения. CollectionModel хранит фильтр для retry и номер запроса, чтобы поздний ответ старого поиска не заменил новый. Когда страница короче лимита, hasMore становится false. Это эвристика, которая сохраняет сигнатуру Result<List<T>>.

10 Сбой сети и повторная попытка

В интеграционном сценарии тестовый контроллер действительно останавливает HTTP-сервер. Затем каталог выполняет новый запрос и показывает ErrorView. После запуска того же сервера кнопка «Повторить» восстанавливает список из SQLite.

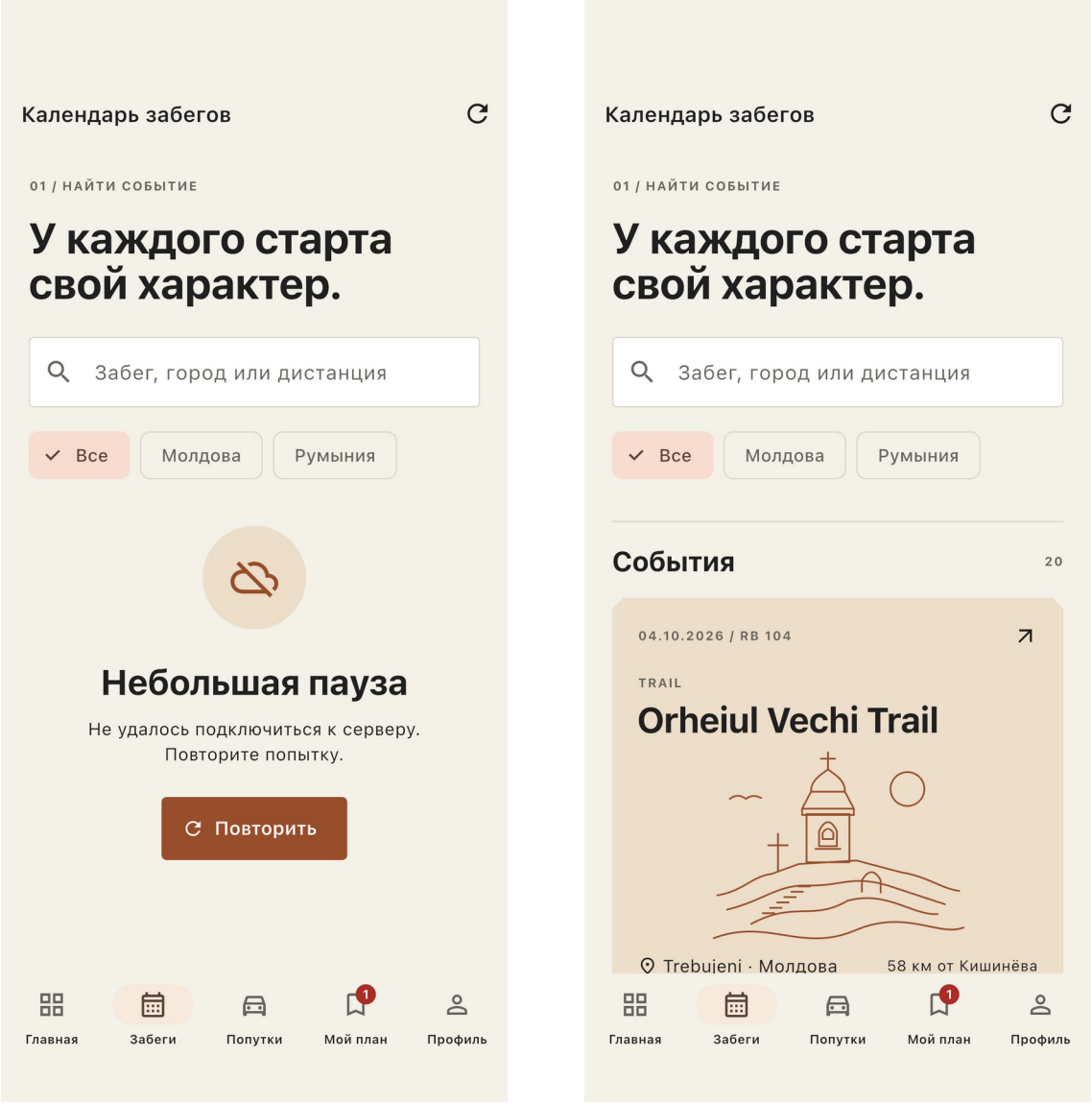


Рисунок 5. Недоступный сервер и успешная повторная загрузка.

Л5 не содержит offline-кэша данных на диске. При неудачной загрузке экран показывает ошибку. Сохранённый JWT служит пропуском, а не копией каталога. При ошибке записи модель не добавляет выдуманный результат и форма сообщает о неудаче.

11 Восстановление token и ответ 401

После полного завершения процесса Simulator и нового запуска обычной сборки main.dart TokenStore читает auth_token из shared_preferences. Главная открывается без входа, участие сохранено. В отдельном этапе теста меняется секрет сервера: подпись старого JWT больше не подходит, первый защищённый запрос возвращает 401.

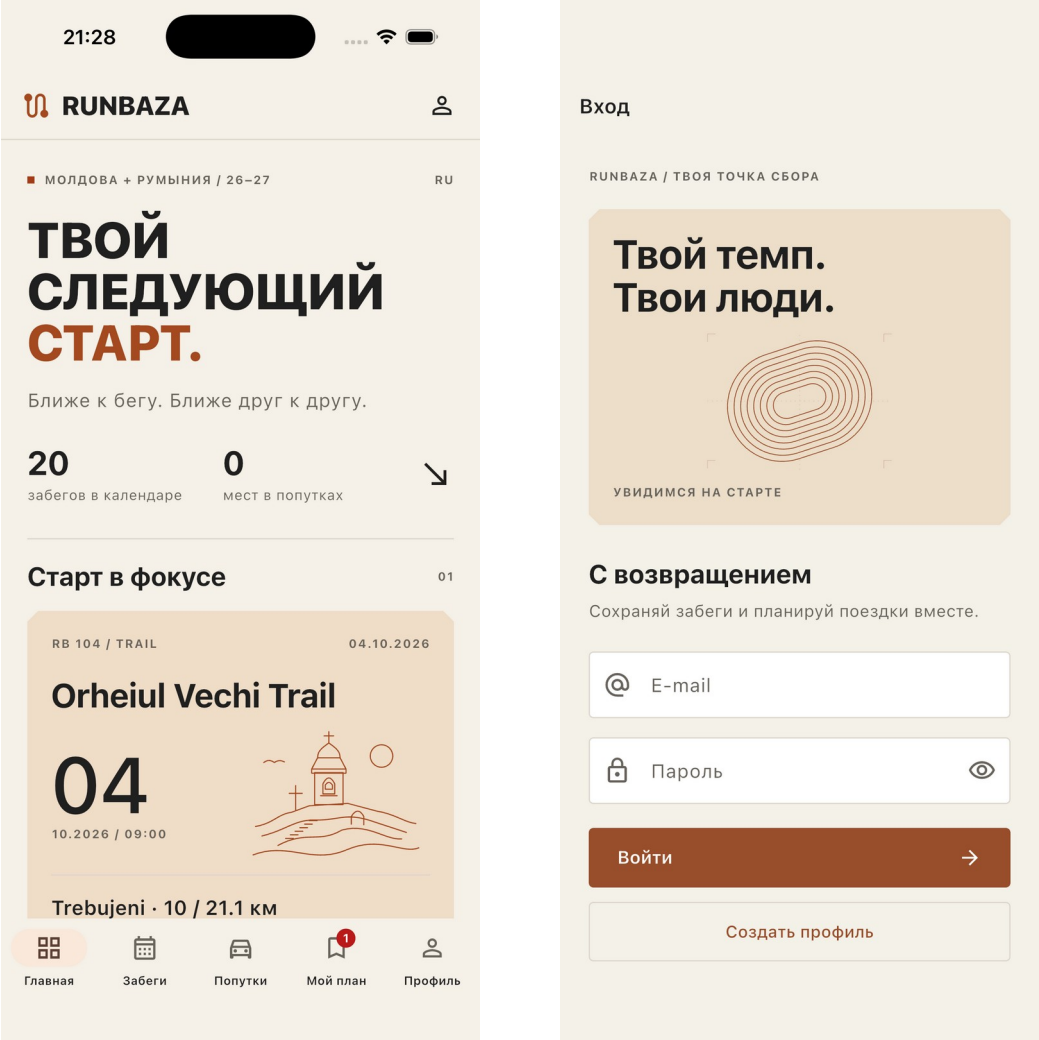


Рисунок 6. Восстановленная сессия и автоматический возврат на вход после 401.

```
if (await tokens.read() != sentToken) return;
await tokens.clear();
onUnauthorized?.call();
```

Сравнение защищает новый token от позднего 401 старого запроса. ApiClient не импортирует router; callback вызывает SessionController, а маршрутизация реагирует на смену состояния. Logout также удаляет token. Пароль не сохраняется локально.

12 Доказательство замены источника

База сравнения Л4: 4c687e9. Коммит fa96d09 фиксирует замену mock на HTTP до добавления пагинации. Команда выполняется из корня РАМ:

```
git diff --stat 4c687e9 fa96d09 -- \
  Lab2/run_baza/lib/screens Lab2/run_baza/lib/models

 Lab2/run_baza/lib/screens/login_screen.dart    | 67 ++++++++++++++++++------
 Lab2/run_baza/lib/screens/register_screen.dart | 55 ++++++++++++-----
 Lab2/run_baza/lib/screens/settings_screen.dart | 47 ++++--------------
 3 files changed, 107 insertions(+), 62 deletions(-)
```

В списке только login_screen.dart, register_screen.dart и settings_screen.dart. В данном проекте login и register представлены отдельными экранами, поэтому исключение auth занимает два файла. Профиль изменён из-за удаления диагностики и настоящего logout. Экраны данных и модели отсутствуют.

Почему итоговый diff больше

Следующий коммит e39b77f добавляет новую функциональность: параметры запроса, состояние пагинации и поиск в каталоге. Эти изменения относятся к заданию 6. Смешивать их с доказательством задания 5 нельзя: на checkpoint уже работали HTTP и формы при прежних моделях.

Проверки Flutter и iOS

flutter analyze проверяет весь проект; flutter test выполняет 147 unit/widget-тестов. Среди них обработка timeout и некорректного JSON, 204, чужой 401 после нового входа, auth, сериализация, гонки поиска и повтор страницы. Интеграционный сценарий использует настоящий Hono API и iPhone Simulator.

```
cd Lab2/run_baza
flutter analyze
flutter test

# With the isolated test API running:
flutter drive --driver=test_driver/lab5_driver.dart \
  --target=integration_test/lab5_test.dart \
  --dart-define=API_BASE_URL=http://127.0.0.1:8090 \
  -d <iphone-simulator-id>
```

13 История разработки

Полный git log --oneline от первого коммита Л2 до версии кода Л5 перед сборкой отчёта. Снимок истории хранится в Lab5/evidence/git-history.txt.

```
d58f31f Lab 5: verify auth and failure recovery on iOS and add launch scripts
e39b77f Lab 5: connect server search and pagination and verify the API in CI
fa96d09 Lab 5: replace runtime mocks with HTTP repositories and real authentication
24e8175 Lab 5: implement Hono API with SQLite, JWT and RunBaza resources
849ad5c Lab 5: add HTTP transport and token storage with error handling
4c687e9 Lab 4: add report, defense guide and iOS evidence
c963eb7 Lab 4: document state ownership and the iOS demonstration
1c7bbf4 Lab 4: verify cross-screen updates and error recovery on iOS
01654aa Lab 4: reuse profile defaults and show pending plan changes
a585b69 Lab 4: keep seed data inside mock repositories
da66016 Lab 4: update form and layout tests for repository-backed screens
047358f Lab 4: redesign the runner profile and save personal preferences
ae0d1c7 Lab 4: guard form callbacks after asynchronous writes
d431219 Lab 4: create driver offers and passenger requests from forms
b929823 Lab 4: load the transfer directory through the catalog
90ee237 Lab 4: save participation forms through the plan repository
998d829 Lab 4: share plan counts and confirm removal across screens
15922f9 Lab 4: read distance pages through the catalog model
```

14 История разработки продолжение

Полный git log --oneline от первого коммита Л2 до версии кода Л5 перед сборкой отчёта. Снимок истории хранится в Lab5/evidence/git-history.txt.

```
c75ddd0 Lab 4: read race details through the catalog model
b803cb8 Lab 4: add catalog loading, errors and retry
1d3a55d Lab 4: load the home screen from shared repository state
2fbefd5 Lab 4: provide shared models above the router and add retry views
7af90f7 Lab 4: model shared state with loading and mutation notifications
b31f9b3 Lab 4: add repository contracts and delayed mock CRUD
43d8db8 Lab 4: add provider dependency
03c3974 Use English in lab documentation and code comments
cdc6c65 Lab 3: add registration form and session profile
f55bdd8 Lab 3: add Word and PDF reports, defense guide, and iOS screenshots
8f003bc Lab 3: keep all form fields mounted while scrolling
cf74c2a Lab 3: scroll long forms in the iPhone integration scenario
3997ef0 Lab 3: add navigation and form tests, iOS scenarios, and launch guide
4f33fc4 Lab 3: add validated forms, profile editing, and logout confirmation
4eab849 Lab 3: add five independent StatefulShellRoute tabs
5c420fe Lab 3: define routes and ID-based navigation
6ffd49a Lab 3: add go_router dependency
c2b88af Add Lab 2: RunBaza interface and animated distance screens
```

Источники

Задание: L5_Integrarea_cu_backendul_RU.pdf, разделы 3–10.

Hono для Node.js: https://hono.dev/docs/getting-started/nodejs

JWT в Hono: https://hono.dev/docs/helpers/jwt

SQLite Node.js: https://nodejs.org/docs/latest-v24.x/api/sqlite.html

Flutter token store: https://pub.dev/packages/shared_preferences

Вывод

RunBaza использует настоящий HTTP и постоянную серверную базу, сохраняя разделение UI, состояния и репозиториев. Проверены auth, CRUD, серверный поиск, страницы, ошибка сети и очистка token. Следующий этап может добавить offline-кэш; в текущей работе он намеренно отсутствует.